EventHub

Event and Participant Management System

Technical Report

Web Programming — Master 1, 2026

Integrated Full-Stack Application

Name: Yacine Benfatma

April 10, 2026

Contents

1 Executive Summary **2**

2 Technology Stack **2**

3 Architecture and Data Model **2**
3.1 High-Level Request Flow 2
3.2 Key Architectural Decisions 3
3.3 Data Model 3

4 Django REST API **3**
4.1 Endpoints 4
4.2 Permissions and Validation 4

5 React SPA Frontend **5**
5.1 Pages and Components 5
5.2 Integration and UX 5

6 Django vs Node.js — Comparative Analysis **5**
6.1 Structure, Philosophy, and Development Complexity 5
6.2 Data Integrity, Scalability, and Ecosystem 6
6.3 Conclusion 6

7 Deployment **6**

8 Testing, Limitations, and Conclusion **7**
8.1 Testing 7
8.2 Limitations and Improvements 7
8.3 Conclusion 7

1 Executive Summary

EventHub is a complete full-stack application for managing events, participants, and registrations. It is built around three integrated layers: a primary **Django REST API**, a **React SPA**, and a comparative **Node.js/Express** backend. The system fulfils all assignment requirements: event and participant CRUD with date/status filtering; a many-to-many registration relationship via a dedicated `Registration` model with uniqueness enforcement; JWT-based authentication with `viewer` (read-only) and `editor/admin` (full write) roles; and production deployment on Render and Vercel.
Repo available at : https://github.com/Yacin-e/projet-web2
Web available at : https://projet-web2.vercel.app/login

2 Technology Stack

Layer	Technology	Rationale
Primary API	Django 5 + DRF + SimpleJWT	Mature ORM, built-in admin, native JWT integration, declarative RBAC.
Comparative API	Node.js 20 + Express + SQLite	Minimal framework that highlights architectural contrast with Django.
Frontend	React 19 + TypeScript + Vite	Component-based SPA with strong typing; Vite for fast builds.
HTTP client	Axios	Response interceptor for transparent 401/token-refresh handling.
Database (dev)	SQLite	Zero-configuration; sufficient for development and demo.

3 Architecture and Data Model

3.1 High-Level Request Flow

The React SPA is served statically from Vercel. On login it calls `POST /api/auth/login/` on the Render-hosted Django API and stores the returned `access` and `refresh` JWT tokens in `localStorage`. Every subsequent request carries `Authorization: Bearer <access>`. A `GET /api/me/` call at startup returns the user role, which the SPA uses to show or hide write controls — a UX complement to server-side permission enforcement. The Node/Express backend runs independently on a separate port and is not used by the

React frontend in production.

3.2 Key Architectural Decisions

[Explicit join model] Rather than using Django's implicit `ManyToManyField`, a dedicated `Registration` model is used, providing a `registered_at` timestamp and a compound `unique_together` constraint on (`event`, `participant`) — directly satisfying the assignment requirement.

[`UserProfile` and role storage] Roles (`viewer`, `editor`, `admin`) are stored in a `UserProfile` linked to Django's built-in `User` via `OneToOneField`. A `post_save` signal creates a default-`viewer` profile for every new user, keeping role management decoupled from the auth model.

[Token refresh interceptor] The Axios instance carries a response interceptor that silently refreshes the access token on any `401` and retries the original request, preventing session interruptions without adding logic to individual API call sites.

3.3 Data Model

Entity	Key Fields	Notes
`Event`	`title`, `start_at`, `end_at`, `status`	Status: `draft|scheduled|ongoing|completed|c` `end_at > start_at` enforced in serializer.
`Participant`	`first_name`, `last_name`, `email`, `phone`	`email` unique; `phone` optional.
`Registration`	`event` (FK), `participant` (FK), `registered_at`	Unique constraint on (`event`, `participant`); auto-timestamp.
`UserProfile`	`user` (1:1), `role`	Roles: `viewer` (default), `editor`, `admin`.

4 Django REST API

4.1 Endpoints

Path	Description	Write restriction
`POST /api/auth/login/`	Returns `access` + `refresh` tokens.	—
`POST /api/auth/refresh/`	Issues a new `access` token.	—
`POST /api/auth/logout/`	Blacklists the `refresh` token.	—
`GET /api/me/`	Current user identity and role.	—
`GET/POST /api/events/`	List (supports `?status=`, `?date_from=`, `?date_to=`) and create.	editor / admin
`GET/PUT/DELETE /api/events/{id}/`	Retrieve (includes registered participants), update, delete.	editor / admin
`GET/POST /api/participants/`	List and create participants.	editor / admin
`GET/PUT/DELETE /api/participants/{id}/`	Retrieve, update, delete.	editor / admin
`GET/POST /api/registrations/`	List (filterable by `?event=` or `?participant=`) and create.	editor / admin
`DELETE /api/registrations/{id}/`	Unregister a participant from an event.	editor / admin
`GET /api/dashboard/`	Aggregate counts: events by status, participants, registrations.	—

4.2 Permissions and Validation

A custom DRF permission class enforces: `GET` open to any authenticated user; `POST/PUT/DELETE` restricted to `editor` or `admin` (or Django superuser), otherwise `403 Forbidden`. Key validation rules:

- **Event:** `end_at > start_at` checked in `validate()`.
- **Registration:** duplicate detection runs in the serializer before the database unique constraint is reached, returning a descriptive `400` rather than an unhandled integrity error.

5 React SPA Frontend

5.1 Pages and Components

The SPA ships five pages: **LoginPage** (credential form, token storage), **DashboardPage** (summary cards, status breakdown), **EventsPage** (list with date/status filters, CRUD for privileged roles), **EventDetailsPage** (event data, registered participants, register/unregister), and **ParticipantsPage** (participant CRUD). All are functional components using `useState`, `useEffect`, and `useMemo`. Shared logic lives in `Layout.tsx` and `ProtectedRoute.tsx`.

5.2 Integration and UX

All HTTP calls pass through `src/lib/api.ts`, an Axios instance with the API base URL, `Authorization` header injection, and the 401-refresh interceptor. Tokens are stored in `localStorage`. Every data-fetching page handles three explicit states: *loading* (spinner), *error* (visible card with the server message), and *role-aware* (write controls hidden for `viewer` accounts — not merely disabled). Unauthenticated users are redirected to `/login` by `ProtectedRoute`.

6 Django vs Node.js — Comparative Analysis

The Node/Express backend implements Event and Participant CRUD with SQLite persistence and basic error middleware. It is used exclusively for comparison; the React frontend does not interact with it in production.

6.1 Structure, Philosophy, and Development Complexity

Django is **convention-driven**: `models.py`, `serializers.py`, `views.py`, and versioned migrations are expected locations, which enforces consistency and accelerates onboarding. Express is **composition-first**: structure is a team decision, reducing boilerplate for small APIs but risking inconsistent patterns at scale.

Aspect	Django + DRF	Node.js + Express
CRUD speed	Very fast via serializers + viewsets + routers.	Fast for small APIs; consistency needs extra patterns.
Validation	Centralised in serializers; model constraints as safety net.	Must select and wire a library (Joi, Zod) per route.
Auth & roles	`simplejwt`: issuance, refresh, blacklisting built-in.	`jsonwebtoken` covers signing; RBAC and blacklisting are custom.
Error handling	DRF returns consistent HTTP codes and JSON payloads by default.	Custom middleware must normalise responses and map DB errors to 4xx.
Migrations	Auto-generated; schema evolution tracked and reproducible.	Requires Prisma, Sequelize, or manual SQL scripts.

6.2 Data Integrity, Scalability, and Ecosystem

Django's `Registration` model carries a `unique_together` constraint; the serializer catches duplicates before the DB is reached. In Node, a SQL `UNIQUE` constraint handles this, but the resulting `SQLITE_CONSTRAINT` error must be manually caught and mapped to `409` — functional, but harder to audit centrally.

Django scales horizontally in a stateless configuration (Gunicorn, Redis, Celery) with a mature Python ecosystem. Node.js excels at I/O-bound and real-time workloads via its non-blocking event loop, with a vast npm registry. Both scale horizontally; architectural discipline is more critical in Express projects due to the lack of enforced conventions.

6.3 Conclusion

For EventHub — a data-centric application emphasising relational modeling, business validation, and RBAC — Django/DRF is the correct primary backend. Node/Express demonstrates that the same CRUD goals are reachable with a lighter framework, while making visible the additional manual work required to achieve equivalent relational safety and auth integration.

7 Deployment

The Django API runs on **Render** (Gunicorn, HTTPS, auto-restart); the React SPA is deployed on **Vercel** (static build, global CDN). All secrets are configured manually through each platform's dashboard — no `.env` files are committed to the repository.

Platform	Variable	Purpose
Render (Django)	`DJANGO_SECRET_KEY`	Cryptographic secret.
	`DJANGO_DEBUG`	Set to `false` in production.
	`DJANGO_ALLOWED_HOSTS`	Render service hostname.
Vercel (React)	`VITE_API_BASE_URL`	Full URL of the Render Django API.

Production configuration: CORS restricted to the Vercel origin via `django-cors-headers`; static files served by WhiteNoise; `vercel.json` rewrites all paths to `index.html` for client-side routing.

8 Testing, Limitations, and Conclusion

8.1 Testing

API validation was performed via Postman: auth flow (login/refresh/logout), role enforcement (`viewer` receives `403` on all write endpoints), duplicate registration rule (`400` with descriptive message), and date/status filtering (correct result sets). Frontend integration was validated through a full end-to-end demo: login as `editor`, create event and participant, register, apply filters, then log in as `viewer` and verify read-only behavior.

8.2 Limitations and Improvements

Current limitations: no automated tests (serializer unit tests, DRF permission integration tests), no pagination on list endpoints, and no full-text search on event titles or participant names. Planned improvements include OpenAPI documentation via `drf-spectacular`, `pytest-django` and React Testing Library test suites, a `docker-compose` setup for unified local development, and a PostgreSQL migration for production-grade concurrency.

8.3 Conclusion

EventHub demonstrates the end-to-end construction of a production-grade web application within four weeks. The architecture reflects deliberate decisions — an explicit join model, a signal-driven profile system, and a transparent token refresh interceptor — rather than minimal implementations. All grade-critical requirements (deployment, authentication, many-to-many relationship, and business validation) are fulfilled.